PRESENTATION FLOW & DEMO PLAN

POUND × Flatterie
Online Booking System

Online Booking and Reservation Management System · Full-Stack MERN Web Application

Course: BSIT-MWA INF242
Subjects (joint project): Integrative Programming and Technologies · System Integration & Architecture
Target client (sample/reference only): POUND × Flatterie, SM Mall of Asia
Members: Ebdane, Al-Vi-Marie M. · Espiritu, Khalil Manuel M. · Montaño, Lou Giuz M. · Umadhay, Kclei F.

This document is the group's plan for the final presentation: the order of topics, what each slide contains, who presents what, the transitions between speakers, the live demo script, and prepared answers to likely questions. It is not a word-for-word script. Speakers explain in their own words, using the key points listed.

1. At a Glance

There is no fixed time limit, so the durations below are **suggestions** that show how much weight each part has. The whole presentation is planned for about **25 minutes + Q&A**. Most of the time goes to the **live demo**, because both guidelines require the system's workflow to be demonstrated.

> WHY (problem & client) → WHO (users) → HOW IT IS BUILT → SEE IT WORK (one booking, created → completed → archived) → HOW WE SOLVED THE HARD PARTS → PROOF EVERY REQUIREMENT IS MET

§	Section	Speaker	Slides	Suggested	Purpose
1	Opening	Al-Vi Ebdane	1–5	3–4 min	Problem, target client, objectives, scope, user roles
2	Technical foundation	Giuz Montaño	6–8	3–4 min	Architecture, RESTful API, database design
3	Demo A: Accounts & Users	Kclei Umadhay	(live)	5–6 min	Modules 1–2: Account Management, User Management
4	Demo B: Reservations & Archive	Khalil Espiritu	9 + (live)	6–7 min	Modules 3–4: Core Transaction (creation → completion), Archive
5	Engineering & critical thinking	Giuz Montaño	10–11	3 min	Concepts applied, problems solved, optimization
6	Closing	Al-Vi Ebdane	12–14	2 min	Requirements checklist, future work, conclusion, GitHub link
	Q&A	Everyone	(appendix)	open	Questions routed to the member who covered that topic

> **Why this order:** the demo follows the real life of one reservation, from a customer creating it, to staff completing it, to archiving it. The System Integration & Architecture guideline asks for exactly that ("the actual workflow of the system from creation through completion"). Concept slides are kept short and placed around the demo, so every explanation is immediately shown working.

> **Speaker assignments are a fair default** (each member leads one or two clear parts). Swap freely so that whoever built a part presents it.

2. Requirements Coverage Map

Both subjects grade the same presentation, so every item from both guidelines is mapped to a place in the flow.

System Integration & Architecture: required modules

Requirement	How we prove it	Where
1. Account Management (registration, login, logout, profile, password recovery)	Live: register (with validation errors), login, edit profile, logout, forgot/reset password	§3 Demo A
2. User Management (view users, add/update, roles & permissions)	Live as Admin: list of 12 users, add a staff account, deactivate → login blocked. Roles slide.	§1 slide 5, §3 Demo A
3. Core Transaction (20+ completed, creation → completion)	Live: booking created (Pending) → Confirmed → Completed. Filter shows 24 Completed + 2 archived = 26.	§4 Demo B
4. Archive (separated, view, restore, demonstrated)	Live: archive a finished booking → it leaves the main list → Archive page → search → restore	§4 Demo B
5. 10+ realistic users with roles	12 accounts: 1 admin, 2 staff, 9 customers (realistic names, emails, PH mobile numbers)	§3 + slide 12
6. Data cleanup before presentation	`npm run seed` on the morning of the presentation: removes test data, loads only the prepared demo data	Slide 12
7. Target client	POUND × Flatterie, SM Mall of Asia (sample/reference organization)	Slide 2

Integrative Programming and Technologies: requirements & rubric

Requirement / criterion	How we prove it	Where
MERN integration + RESTful APIs	Architecture diagram; API route groups with methods and status codes	Slides 6–7
Authentication	bcrypt password hashing, JWT login token, role guards; shown live (wrong password, inactive account blocked)	Slide 10 + demo
CRUD operations	Create/read/update on users, tables, reservations; archive instead of delete	Slide 10 + demo
Database management	MongoDB Atlas, 3 collections, references, index	Slide 8
Input validation & error handling	Validation messages in the demo; one central error handler with clear status codes	Demo + slide 10
Responsive UI	Phone layout with the ☰ menu (browser device mode)	Demo A step 1 + slide 10
Modular programming, reusable components, project structure	routes → controllers → models; reusable components and hooks	Slide 10
Rubric: **Focus**	Clear problem, client, users and objectives	§1

Rubric: **Content / Subject knowledge**	Confident explanation of the stack and concepts	§2, §5
Rubric: **Critical thinking**	Problems solved, debugging, optimization	Slide 11
Rubric: **Organization**	Logical flow of the talk and of the app; organized code structure	Whole flow
Rubric: **Writing conventions**	No typos on slides; commented, readable code	All slides (see §6 fixes)
Submission: GitHub link	Repository link / QR on the final slide	Slide 14

3. Section-by-Section Plan (slides, key points, transitions)

§1 Opening Al-Vi Ebdane 3–4 min

Slide	What appears on the slide	Key points to say
1. Title	Project name, course, both subjects, 4 members (current design is fine)	Greet the panel; name the project and the group.
2. Problem & Target Client	POUND × Flatterie, SM Mall of Asia (sample client, not affiliated). The problem: reservations handled manually are less organized, harder to track, and less convenient.	Why a restaurant needs online booking; why this client fits.
3. Objectives	Current objectives (typo fixed) + 2 staff-side objectives: *prevent double booking through automatic table assignment*; *keep records organized with an archive*.	One general objective, then the specific ones.
4. Scope & Limitations	**In scope:** public website, online booking, 4 required modules, one branch. **Out of scope:** online payment, delivery, chat, inventory, multiple branches. **Note:** password-reset email is simulated (link shown in the server console).	Sets clear boundaries before the demo, so no surprises.
5. Users & Roles	3 columns: **Customer** (browse, register, book, view/cancel own bookings) · **Staff** (manage reservations, tables, customers, archive) · **Admin** (everything staff can do + manage user accounts)	Who uses the system and what each role is allowed to do.

Al-Vi → Giuz: "*Those are the three kinds of users our system serves. Giuz will now show how we built the system behind them.*"

§2 Technical Foundation Giuz Montaño 3–4 min

Slide	What appears on the slide	Key points to say
6. System Architecture	Diagram: **React (Vite)** frontend ↔ **REST API (Express + Node.js)** ↔ **MongoDB Atlas** (via Mongoose). Login token (JWT) sent with each request.	How the 4 MERN parts connect; frontend and backend are separate apps that talk through the API.
7. RESTful API	6 route groups: `/auth`, `/users`, `/tables`, `/reservations`, `/customers`, `/archive`; methods GET/POST/PUT/PATCH; status codes 200, 201, 400, 401, 403, 404, 409.	One example request end to end (e.g. booking: POST /api/reservations → 201 Created).
8. Database Design	3 collections: **users** (name, email, phone, password hash, role, status, archive fields), **tables** (number, capacity, location, status), **reservations** (customer → users, table → tables, date, time, guests, status, cancellation and archive fields). Index on table + date + time.	References instead of copies; why the index helps the conflict check.

Giuz → Kclei: "*That is the structure. Now let's see it from the user's side. Kclei will start as a customer visiting the website.*"

§3 Demo A: Accounts & Users (Modules 1–2) Kclei Umadhay 5–6 min

Live demo. Full click-by-click steps are in **§4 Live Demo Script**.

> **Kclei → Khalil:** "*The booking we just made is now* **Pending**, *waiting for the restaurant. Khalil will take it from here as a staff member.*"

§4 Demo B: Reservations & Archive (Modules 3–4) Khalil Espiritu 6–7 min

Slide	What appears on the slide	Key points to say
9. Reservation Life Cycle	Status diagram: **Pending** → **Confirmed** → **Completed**; Pending/Confirmed → **Cancelled** (by customer, rejected by staff, or no-show). 5 two-hour seatings (11 AM, 1, 3, 5, 7 PM). Conflict rule: one active booking per table per seating.	Explain the rules first, then show them live. Completed and Cancelled are final.

Then live demo (see §4).

> **Khalil → Giuz:** "*Behind those buttons are a few problems we had to solve. Giuz will walk through the main ones.*"

§5 Engineering & Critical Thinking Giuz Montaño 3 min

Slide	What appears on the slide	Key points to say
10. Core Concepts Applied	**Authentication:** bcrypt hashing, JWT, role guards (`protect`, `allowRoles`). **Validation:** on the website and again on the server. **Error handling:** one error handler, clear messages and status codes. **CRUD** on all 3 collections. **Responsive:** phone layout and ☰ menu. **Modular:** routes → controllers → models; reusable components (Modal, forms, password field) and hooks.	Link each concept to something the panel just saw in the demo.
11. Problems We Solved	Pick 3–4: **(a)** double booking → conflict check + automatic smallest-table assignment; **(b)** editing a booking "conflicted with itself" → the check ignores the booking being edited; **(c)** data disappeared on every restart → moved from in-memory data to MongoDB; **(d)** speed: free tables found with 2 database queries, the 5 time slots checked in parallel, plus an index; **(e)** safe data cleanup: the seed script validates the data before loading it.	Describe each as problem → cause → solution. This is the "Critical thinking" criterion.

> **Giuz → Al-Vi:** "*To close, Al-Vi will show how all of this meets the project requirements.*"

§6 Closing Al-Vi Ebdane 2 min

Slide	What appears on the slide	Key points to say
12. Requirements Checklist	✓ all 7 required items with proof: Account Mgmt · User Mgmt · Core Transaction (**26 completed**) · Archive · **12 users** (1 admin, 2 staff, 9 customers) · Data cleanup (seeded this morning) · Target client	Quick recap; everything was just shown live.

13. Future Work	Real email/SMS notifications, online deposit/payment, multiple branches, admin-editable menu, reports and analytics.	Ideas for improvement (required in the documentation too).
14. Conclusion	Current conclusion text + "Thank you" + **GitHub repository link or QR code**.	Thank the panel; invite questions.
Appendix	Backup screenshots of every demo step.	Only used if the live demo fails.

4. Live Demo Script

Demo accounts

Role	Name	Email	Password	Used by
Admin	Giuz Montaño	`giuz.montano@gmail.com`	`Admin@123`	Kclei (users), Khalil (optional account archive)
Staff	Rene Baterbonia	`rene.baterbonia@gmail.com`	`Staff@123`	Khalil
Staff	Marco Delos Reyes	`marco.delosreyes@gmail.com`	`Staff@123`	Backup
Customer	Maria Santos (+ 8 others)	`maria.santos@gmail.com`	`Customer@123`	Backup
Customer (inactive)	Paolo Garcia	`paolo.garcia@outlook.com`	`Customer@123`	Shows "deactivated" login block

> **Tip:** use one laptop and separate browser windows so nobody has to log out mid-demo: normal window = customer, InPrivate/Incognito window = staff, a second browser = admin.

Demo A: Accounts & Users Kclei

#	Account	Do	Panel sees	Proves
1	Logged out	Short tour of the home page: hero, menu carousel (click a dish), Visit Us (map, "Open now"). Optionally show phone view with the ☰ menu.	Polished, responsive website	Responsive UI
2	—	Register with an invalid email or a short password, then correct it (e.g. a new customer "Bea Reyes").	Validation messages, then account created and logged in	Registration, validation
3	New customer	Book a Table: tomorrow, change guests (2 → 6 → 10) to show "*Only … tables left*" / "*Last table!*", then book 2 guests at an available time.	Success message with table number and **Pending** status	Transaction created
4	New customer	My Reservations (Upcoming, Ref #) → My Profile (edit name) → Logout	Booking listed; name updated; logged out	Profile, logout
5	—	Forgot Password with the new email → open the link shown in the server console → set a new password → log in	Password reset works; link is one-time	Password recovery
6	Paolo	Try to log in as Paolo	"This account has been deactivated…"	Access control
7	Admin (Giuz)	Users: 12 accounts, filter by role → Add User (staff) → Deactivate it → its login is blocked	Account list and management	User management, roles

Demo B: Reservations & Archive Khalil

#	Account	Do	Panel sees	Proves
0	Slide 9	Explain the life cycle and the conflict rule	Status diagram	Core process
1	Staff (Rene)	Overview → Reservations → **Confirm** the booking Kclei made	Today's numbers; status changes to Confirmed	Workflow step
2	Rene	+ New Booking (walk-in) for the **same date, time and party size** → open the Table list	The table already taken is **not offered**	Conflict prevention
3	Rene	Yesterday's Confirmed booking (Syd Villanueva) → **Complete** (or No-show with a reason)	Status Completed	Creation → completion
4	Rene	Status filter: Completed	24 visible (+2 in Archive = 26)	20+ completed
5	Rene	Tables → Daily Schedule; Customers → View one customer's history	Bookings per table/seating; visit history	Records management
6	Rene	**Archive** a Completed booking → open Archive page → search its name or #ref → **Restore**	Leaves the main list; found in Archive; restored unchanged	Archive (required)
7	Admin (optional)	Users → Archive inactive Paolo → Archive → User Accounts → Restore	Accounts can be archived too	Archive (bonus)

> **If the live demo fails** (no internet, MongoDB blocked by school Wi-Fi): switch to a phone hotspot first. If it still fails, use the appendix screenshots or a pre-recorded demo video, and keep explaining the same steps.

5. Preparation Checklist

Before presentation day

- ☐ Revise the slides following §3 and fix the items in §6.
- ☐ Push the final code to GitHub; every member pulls and runs `npm install` in both `server` and `client`.
- ☐ Change `JWT_SECRET` in `server/.env` to a new random value (everyone gets logged out once; that's expected).
- ☐ Rehearse the full flow at least once, including the speaker transitions.
- ☐ Take backup screenshots of every demo step; optionally record a short demo video.
- ☐ Finish and submit the Technical Documentation (see §6).

Presentation morning

- ☐ Tell the group, then run `npm run seed` in `server` (data cleanup; dates line up with that day). Expect: 12 users, 40 reservations, Completed: 26.
- ☐ Start backend (`npm run dev` → "MongoDB connected") and frontend (`npm run dev`).
- ☐ Open the demo windows (customer / staff / admin) and test one login each.
- ☐ Clear saved passwords in the demo browser so autofill doesn't show logins.
- ☐ Bring a phone hotspot and the demo accounts written on paper.
- ☐ Keep the server terminal visible for the password-reset link.

6. Fixes Needed & Gaps

Statements in the current deck that do not match the system

Slide	Current	Change to
Features	Menu "provides a link to the full menu"	"Shows the dishes in a moving carousel and a list with prices; click a dish for details."
Features	"Analytics & Reporting": allows customers to sign up or log in	Rename to **"Login / Sign Up"**. There is no analytics module; move "reports and analytics" to Future Work.
Reservation flow	"Reservation Completed: view under My Reservations"	**"Reservation Submitted (Pending)"** + a step "Staff confirms → Confirmed". In the system, *Completed* means the guests already dined.

Typos (graded under Writing Conventions)

Slide	Current	Fix
Objectives	"o develop a user-friendly website…"	"To develop a user-friendly website…"
Target Users	"TARGE USERS"	"TARGET USERS"
Target Users	"Benefts"	"Benefits"

Conclusion	Header "Conclusiom"	"Conclusion"

Missing slides to add

Scope & Limitations (4), Users & Roles (5), System Architecture (6), RESTful API (7), Database Design (8), Reservation Life Cycle (9), Core Concepts (10), Problems We Solved (11), Requirements Checklist (12), Future Work (13), GitHub link (14).

Other deliverables

- **Technical Documentation** (separate submission, IPT format): cover page, table of contents, overview, scope, system architecture, database design, UI screenshots, feature list, summary, future work, sources. Headings 14 pt, body 11 pt, 1.5 spacing, 1-inch margins.
- **Sources/References** to list: React, Vite, React Router, Node.js, Express, MongoDB Atlas, Mongoose, bcryptjs, jsonwebtoken, cors, dotenv, nodemon, GSAP, Google Maps embed, Google Fonts.
- **GitHub repository link**: `github.com/CodeLil011/Online-Booking-System` (check it's accessible to the professors).

7. Anticipated Questions & Answers

Answers are based on how the system actually works. Route each question to the member who presented that topic.

Question	Answer	Who
Who is your target client and why?	POUND × Flatterie at SM Mall of Asia, used as a sample/reference organization as the guideline allows. A busy mall restaurant benefits from organized online reservations. The website states it is a student project and not affiliated.	Al-Vi
What is out of scope?	Online payment, delivery, chat, inventory, and multiple branches. The password-reset email is simulated.	Al-Vi
How do the frontend and backend communicate?	The React app calls the Express REST API over HTTP with JSON. After login, the JWT token is sent in the Authorization header of each request.	Giuz
Where is the data stored?	MongoDB Atlas (cloud), through Mongoose models: users, tables, reservations. A reservation stores references (IDs) to its customer and table.	Giuz
How are passwords stored?	Never as plain text: hashed with bcrypt. Login compares the hash. The hash is never sent to the browser.	Giuz
How do roles and permissions work?	Each request passes two checks: `protect` (valid token, account still active) and `allowRoles` (e.g. only admin for user management). The website also hides pages a role can't use, but the server is what enforces it.	Giuz / Kclei
Is the password-reset email real?	Simulated for this project: the link appears in the server console. The token is random, stored only as a SHA-256 hash, expires in 15 minutes, and works once.	Kclei
Can staff create admin accounts or edit users?	No. Only the admin manages accounts. The admin can't change their own role or deactivate themselves, and there must always be at least one active admin.	Kclei
How do you prevent double booking?	Before saving, the server checks for any Pending or Confirmed booking on the same table, date and time (cancelled ones don't count). Customers can't pick tables: the smallest free table that fits is assigned automatically. Staff only see free tables, and the server checks again on save.	Khalil
Why fixed 2-hour seatings?	Bookings in different seatings never overlap, which keeps the conflict rule simple and reliable, like a restaurant's seating schedule.	Khalil
What is a "completed transaction" in your system?	A reservation that went through the workflow until the guests dined and staff marked it Completed. The demo data has 26.	Khalil
Why archive instead of delete?	Records must be kept but separated from active ones. Archiving marks the record and saves who archived it and when, so it can be restored. Only finished bookings and inactive accounts can be archived.	Khalil

What if two people book the last table at the same moment?	The server re-checks availability when saving, so the second request is refused. A database-level lock would be a stronger future improvement.	Khalil / Giuz
How did you prepare the 20 transactions and 10 users?	A seed script loads realistic demo data (12 users, 40 bookings, 26 completed), with dates counted from the day it runs. It first checks the data with the system's rules, and running it also performs the data cleanup.	Al-Vi / Giuz
Is it responsive?	Yes. Layouts adapt to tablet and phone widths, including a ☰ menu. Can be shown live with the browser's device mode.	Kclei
What was the hardest bug?	Pick a real one from slide 11, e.g. editing a booking conflicted with itself, solved by excluding that booking from the conflict check.	Giuz
Did you use AI tools?	Answer honestly and in line with your course policy: what you used them for and what the group did itself.	Everyone

Prepared for the group's internal use. Speaker assignments and suggested durations can be adjusted.